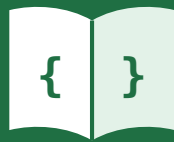

Module 12 — Security with Spring Security

Exercises

Spring Boot Edu



2026-09-24

Contents

How to Work	2
Exercise 1 — Role-Based Access (Easy)	2
Exercise 2 — Only Your Own Orders (Medium)	3
Exercise 3 — Roles from JWT Claims (Hard)	3
Extra Challenge (Optional)	4

How to Work

1. The starter code is in `modules/12-security/exercise/`. Find the TODO comments.
2. The exercises need no Docker: the users live in memory, and the tests sign their own JWTs.
3. Run the tests:

```
./mvnw -Pexercises -pl modules/12-security/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/12-security/solution/`.

★ Tip

The package given contains the users (ada, bob as CUSTOMER, admin as ADMIN), the API controllers and a signing key. You do not need to change it.

Exercise 1 — Role-Based Access (Easy)

Goal: protect the shop API with URL rules for two roles.

Request	Who may call it
GET /api/catalog	everybody, also without login
POST /api/catalog	ADMIN
/api/cart/**	CUSTOMER
/api/admin/**	ADMIN
everything else	every logged-in user

Tasks (`exercisel.ExerciseSecurityConfiguration`):

- TODO 1a — GET /api/catalog is public. POST /api/catalog only for ADMIN.
- TODO 1b — /api/cart/** only for CUSTOMER.
- TODO 1c — /api/admin/** only for ADMIN.

Hints:

- Lesson section 3.3: `.requestMatchers(HttpMethod.GET, "/api/catalog").permitAll(), .hasRole("ADMIN")`.
- The rules are checked **in order**, and the first matching rule wins. `anyRequest()` must stay last.
- An admin is not automatically a customer: `hasRole("CUSTOMER")` rejects the admin.

Acceptance criteria: all 5 tests in Exercise1Test pass.

Estimated time: 20 minutes

Exercise 2 — Only Your Own Orders (Medium)

Goal: customers see only their own orders, admins see all of them. The rules belong on the service, so that they hold for every caller.

Tasks (exercise2.OrderQueries):

- TODO 2a — ordersOf(customer): only the customer themselves or an ADMIN may call it.
- TODO 2b — recentOrders(): remove every order that does not belong to the caller from the returned list.
- TODO 2c — find(id): the returned order may only be seen by its customer or an ADMIN.

Hints:

- Lesson section 3.4: @PreAuthorize, @PostAuthorize and returnObject.
- A method parameter is available in the expression with #: #customer == authentication.name.
- @PostFilter("filterObject.customer() == authentication.name") removes elements from the returned collection. It needs a mutable collection, which recentOrders() already returns.

Acceptance criteria: all 5 tests in Exercise2Test pass.

Estimated time: 30 minutes

Exercise 3 — Roles from JWT Claims (Hard)

Goal: our identity provider puts the user's roles into its own claim, and the readable user name into preferred_username:

```
{ "sub": "user-7", "preferred_username": "ada", "roles": ["customer"], "iss":  
  ↪ "https://id.bookstore.example" }
```

By default, Spring Security takes the name from sub and the authorities from scope. Map the claims so that the rules of exercise 1 work with these tokens.

Tasks (exercise3.JwtRolesConfiguration):

- TODO 3a — a JwtAuthenticationConverter bean. The resource server picks it up automatically.

- TODO 3b — the principal name comes from the `preferred_username` claim.
- TODO 3c — every entry of the `roles` claim becomes an authority `ROLE_<ROLE IN CAP-ITALS>`, e.g. `customer` → `ROLE_CUSTOMER`.

Hints:

- `converter.setPrincipalClaimName(...)`.
- `converter.setJwtGrantedAuthoritiesConverter(jwt -> ...)` receives the `Jwt`. `jwt.getClaimAsStringList("roles")` may be null for a token without the claim.
- `role.toUpperCase(Locale.ROOT)`: without a fixed locale, `"admin".toUpperCase()` on a Turkish system is `"ADMİN"` (dotted İ), and the role check fails.

Acceptance criteria: all 3 tests in `Exercise3Test` pass.

Estimated time: 40 minutes

Extra Challenge (Optional)

The JWTs of exercise 3 are only checked for signature and expiry. Add validators so that tokens are only accepted from the issuer `https://id.bookstore.example` and only if their `aud` claim contains `bookstore-api` (`JwtValidators.createDefaultWithValidators(...)`, `JwtClaimValidator`). Write a test with a token from another issuer.